

this put a value like boolean true/false to n & check in func: if $n == \text{True}$ or False then put $n = \text{classname}()$, so now since it's not in place of default, any func would create a new object time, instead of using same & again & again.

★ Numpy ★

- Its full form is Numerical Python, it's a package for faster & efficient computation & processing of multidimensional elements.
 - As it's written in C, it performs faster than normal python list & other things.
 - used for working with matrices, vectors & statistics.
- ex normal list $[1, 2, 3]$
numpy $\rightarrow$ an array like $\text{np.array}([1, 2, 3])$

It's used for Speed.

ex. $\text{lst} = [1, 2, 3]$

$\text{doubled} = [n * 2 \text{ for } n \text{ in } \text{lst}]$

import numpy as np
 $\text{arr} = \text{np.array}([1, 2, 3])$
 $\text{doubled} = \text{arr} * 2$

$\Rightarrow$ so don't have to use any loops or list comprehension to make doubled list saving space & time.

used for space

ex Import numpy as np

import sys

lst = range(1000)

size = sys.getsizeof(1) * len(lst)

print(sys.getsizeof(1))

print(size)

np1st = np.arange(1000)

size1 = np1st.size * np1st.itemsize

print(np1st.itemsize)

print(size1)

Output

28

28000

8

8000

⇒ `size` is used find size of np array & `itemsize` is used to find size of item in iterable like `np1st` also you can see where python normal integer takes size of 28, np item only takes 8 i.e. less than $\frac{1}{3}$ so meaning it is not only efficient in speed, but it is also efficient in space consumption.

1) ndarray (n-dimensional array)

ex `a = np.array([1, 2, 3])`

`b = np.array([[1, 2, 3], [4, 5, 6]])`

`print(type(b))` → `<class 'numpy.ndarray'>`

`print(b.shape)` $\rightarrow$ (2,3) # row, col.

print(b.size) → 6

$$\text{plant}(\text{b}, \text{ndim}) \rightarrow 2$$

`print(b.dtype)` $\rightarrow$ int64

- `shape` → used to print size of ndarray in row, column.

size $\rightarrow$ used to print no. of items in ndarray.

ndim $\rightarrow$ " " " " " dimension in ndarray.

• dtype → " " " type of stored^o in ndarray.
data

2) Creating arrays

① from python list :-

np.array([1, 2, 3]) $\rightarrow$ [1, 2, 3]

np.array([1, 2, 3], [4, 5, 6]) $\rightarrow$ $\begin{bmatrix} 1, 2, 3 \\ 4, 5, 6 \end{bmatrix}$

② used for creating empty grids $\rightarrow$ zeros, ones, full

np.zeros(2, int) $\rightarrow$ ~~[0 0]~~ [0 0]

$$\text{np.zeros}((2,3), \text{int}) \rightarrow \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

~~S~~ $l = [2, 3]$
 ~~$\text{np.zeros}(l, \text{int})$~~ (wrong)
 (same) $\star$

```
( row = 2  
  col = 3  
  data type = int )
```

`np.ones((2,3),int)` → $\begin{bmatrix} [1 & 1 & 1] \\ [1 & 1 & 1] \end{bmatrix}$ line-ones
line-zeros

`np.full((2,3),7,int)` → $\begin{bmatrix} [7 & 7 & 7] \\ [7 & 7 & 7] \end{bmatrix}$

⇒ for `.full()` have 3 things mainly ⁽²⁾ row, ⁽³⁾ col could also give only row also 7 that is to fill in the matrix as data value, int is type of data in matrix so above line make a matrix of size 2,3 full of item 7 with having datatype 'int'. (If no data type then takes float by default)
ex 1.0 or 7.0 etc.

III Identity Matrix

★ syntax → `np.eye(3,3,0,int)` optional
row colⁿ (k)
tells where to place 1's on type of data in matrix.
 $k=0 \rightarrow$ diagonally
 $k>0 \rightarrow$ above diagonal
 $k<0 \rightarrow$ below diagonal

ex `np.eye(3,3,0,int)` → $\begin{bmatrix} [1 & 0 & 0] \\ [0 & 1 & 0] \\ [0 & 0 & 1] \end{bmatrix}$ If $k=1$ start 1 upper diagonal or start from 1 colⁿ diag.
If $k=-2$ start from 2 lower or 2 row diag.

★ `np.identity(3,int)` → also makes identity matrix of size 3 having int values.

IV) arange & linspace

* arange → returns a array like range

syntax → $\text{np.arange}(1, 10, 2) \rightarrow [1 \ 3 \ 5 \ 7 \ 9]$
start ——— end point ——— step.

→ $\text{np.arange}(5) \rightarrow [0 \ 1 \ 2 \ 3 \ 4]$

if only 1 value prints from 0 to given number.

⇒ in both cases end point is never included like the function range().

* linspace → gives evenly spaced values b/w two numbers

syntax → $\text{np.linspace}(0, 10, 5) \rightarrow [0. \ 2.5 \ 5. \ 7.5 \ 10.]$
start ——— end ——— no. of values

⇒ this linspace by default includes end point but can be changed & also datatype can also be changed.

3) Indexing and Slicing

ex $a = \text{np.array}([10, 20, 30, 40])$

$a[0] \rightarrow 10$

$a[1:3] \rightarrow [20, 30]$

$a[1:4] \rightarrow [20, 30, 40]$

(while printing numpy array comma does not appear like they do in list)

`p = np.array([[1, 2, 3], [4, 5, 6]])`

`print(p[0,0])` → 1

`print(p[1,2])` → 6

`print(p[:,1])` → [2 5]

`print(p[1,:])` → [4 5 6]

Stored as
numpy 2d
array

$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$

⇒ so for n-dimensional numpy array we do indexing or slicing by row, col pair ex `p[1,2]` means row = 1, col = 2 i.e. 6 because indexes always start with 0, also when you encounter (:) colon means take every thing in this indexing, meaning if `p[:,1]` means take whole all row values of 2d array and col = 1 take 1st colⁿ i.e. index = 1 colⁿ so all row values of index 1 colⁿ i.e. [2 5].

Boolean Indexing

`r = np.array([10, 20, 30, 40, 50])`

`mask = r > 20`

`print(mask)` → [false false True True True]

`print(r[mask])` → [30 40 50]

`print(r[r > 20])` → [30 40 50]

`r[r > 20] = 100`

`print(r)` → [10 20 100 100 100]

so in this basically a mask used is a condition which checks on all values on which it's applied & then returns a array of boolean values if they (value) satisfied the condition then in its place True otherwise false, we can apply mask directly on x (given array) & it would then only have those values who satisfied condition, we can apply mask by storing it in a variable ex "mask" or by directly $x[x > 20]$ also we can assign values to those who satisfied condition like 30, 40, 50 then it would change the original array. we can also do $x[x = 20] \rightarrow [\text{false false True false false}]$ only True at value 20.

4) Vectorized Operation

$\rightarrow$ numpy is built to avoid use of 'for' loops.

$a = \text{np.array}([1, 2, 3])$

$b = \text{np.array}([4, 5, 6])$

$\text{print}(a+b) \rightarrow [5 \ 7 \ 9]$

$\text{print}(a-b) \rightarrow [3 \ 3 \ 3]$

$\text{print}(a*b) \rightarrow [4 \ 10 \ 18]$

$\text{print}(b\%a) \rightarrow [0 \ 1 \ 0]$

$\text{print}(a**2) \rightarrow [1 \ 4 \ 9]$

* // for floor-division, a/b then floor of result value

* Also could use .add() .mult() for more control but for normal $+$, $-$ could be used if extra not required

(done without the use of loops)

5) Broadcasting (different array working together)

- ex Numpy can automatically match arrays with different shapes during mathematical operations if one of the dimensions is 1.

ex $a = \text{np.array}([1, 2, 3], [4, 5, 6])$
 $b = \text{np.array}([10, 20, 30])$

$\text{print}(a+b) \longrightarrow \begin{bmatrix} 11 & 22 & 33 \\ 14 & 25 & 36 \end{bmatrix}$

$\Rightarrow$ so here in this in b array one dimension ~~is~~ ^{now} is 1
so python stretched this one row over all rows
in a that's why a+b worked.

- Also for a scalar / single value it works too

ex $c = [20]$
 $\text{print}(a+c)$

$d = 20$

$\text{print}(a+d)$

$\begin{bmatrix} 21 & 22 & 23 \\ 24 & 25 & 26 \end{bmatrix} \xrightarrow{\text{same}}$

- easy rule is if shapes are same or one dimension is 1.

that's why these work $\rightarrow (3), (3) \mid (2,3), (3) \mid (2,3), (1,3)$
(same) (2nd one stretches) (now can stretch)

$(2,3), (1)$
stretch to other's
rows

But $(2,3), (2,2) \rightarrow$ no dim. 1
 $(3) \& (4) \rightarrow$ diff shape
don't work
 $(2,3) \& (4,3,3) \rightarrow$ 1 dim absent & no dimension is 1.

6) Aggregation functions (ex min, max etc)

ex `a = np.array([[1, 2, 3], [4, 5, 6]])`

`print(a.min())` $\rightarrow 1$

`print(a.max())` $\rightarrow 6$

`print(a.sum())` $\rightarrow 21$

`print(a.mean())` $\rightarrow 3.5$

$\Rightarrow$ variance $\rightarrow$ how spread out the numbers are from average
more randomness $\rightarrow$ more variance
low " $\rightarrow$ less variance

$\Rightarrow$ standard deviation $\rightarrow$ how much the numbers differ from average, but on same scale as numbers
if all numbers are same $\text{std} = 0$.

★ $\text{standard deviation} = \sqrt{\text{variance}}$

`print(a.var())` # variance of whole array $\rightarrow 2.916 \dots$

`print(a.var(axis=0))` # variance per colⁿ $[2.25 \quad 2.25 \quad 2.25]$

`print(a.var(axis=1))` # variance per row $[0.6 \dots \quad 0.6 \dots]$

`print(a.std())` $\rightarrow 1.707 \dots$

* Axis Based Aggregate functions

axis - parameter controls direction

• axis = 0 (colⁿ wise)

• axis = 1 (row wise)

ex $a.sum(\text{axis}=0) \rightarrow [5 \ 7 \ 9]$ (colⁿ wise sum)

$a.sum(\text{axis}=1) \rightarrow [6 \ 15]$ (row wise sum)

$a.min(\text{axis}=0) \rightarrow [1 \ 2 \ 3]$

$a.max(\text{axis}=1) \rightarrow [3 \ 6]$

7) Reshaping & Transposing

$a = np.arange(6)$

$b = a.reshape(2, 3)$

$print(b) \rightarrow \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \end{bmatrix}$

reshape() makes a new shaped array by using given dimensions & filling available data in them, if not enough data throws value error.

$c = b.flatten()$
 $= b.ravel()$ $\rightarrow [0 \ 1 \ 2 \ 3 \ 4 \ 5]$

(flatten() & ravel() both work by simply making 1D array from given data ex from b)

*

But, flatten returns a new 1D array but ravel returns a view & might return a copy if view not returned & if ravel returned a new then it also changes original so if want a new use flatten but its slower & less efficient.

$d = b.T$
 $= b.transpose()$

$\rightarrow \begin{bmatrix} [0 & 3] \\ [1 & 4] \\ [2 & 5] \end{bmatrix}$

both $.T$ & $.transpose()$
 returned transpose of b
 But $.T$ is a property &
 $.transpose$ is a method

$\Rightarrow$ $.transpose$ allows to use different order of axes
 but $.T$ does not so $.T$ is used for quick transpose
 & other one might be used for custom transpose
 & both return view of original array not a copy

8) Stacking & Concatenating

Stacking $\rightarrow$ Stacking means putting a array either
 vertically or horizontally alongside each other

$a = np.array([[1, 2], [3, 4]])$

$b = np.array([[5, 6]])$

$print(np.vstack([a, b])) \rightarrow \begin{bmatrix} [1 & 2] \\ [3 & 4] \\ [5 & 6] \end{bmatrix}$

$print(np.hstack([a, a])) \rightarrow \begin{bmatrix} [1 & 2 & 1 & 2] \\ [3 & 4 & 3 & 4] \end{bmatrix}$

$\Rightarrow$ we used a, a in $hstack$ as b can't be stacked
 horizontally to a

`c = np.array([[5], [6]])`

`print(np.hstack([a, c]))` → $\begin{bmatrix} 1 & 2 & 5 \\ 3 & 4 & 6 \end{bmatrix}$

⇒ so b couldn't be stacked to a but c could because b had only 1 row 3 colⁿ not similar to a with 2 row & 2 colⁿ but c ~~have~~ had 2 row & 1 colⁿ which could be stacked to a horizontally by common sense.

Concatenate → concatenate means addition of two things, we can also tell to ~~the~~ concatenate which way.

ex `print(np.concatenate([a, b], axis=0))` → $\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$

⇒ just concatenate takes `axis=0` is concatenate by row it added b to a but if you try `axis=1` on a & b it would show error, also `axis=0` is like ~~vstack~~ & `axis=1` is like ~~hstack~~

⇒ concatenate takes value of axis as 0 by default.
⇒ `concat` `axis=0` is added b to a colⁿ wise.
i.e. adding b below a column wise.

1) Utility functions (unique, sort, clip, where, argsort)

a = np.array([1, 1, 2, 3, 4, 5])

★

print(np.unique(a)) → [1 2 3 4 5] Unique

print(np.unique(a, return_counts=True))

↳ (array([1, 2, 3, 4, 5]), array([2, 1, 1, 1, 1]))

↑
unique values

↑
counts in original.

b = np.array([14, 11, 12, 3, 4, 5])

★

print(np.sort(b)) → [3 4 5 11 12 14]

print(np.argsort(b)) → [3 4 5 1 2 0]

Sort &
argsort

indexes of sorted array
elements in original array
ex 0 is index of 14 in
sorted array from

original array.

c = np.array([10, 20, 30, 40, 50])

index whose
condition is True

print(np.where(c > 25)) → (array([2, 3, 4]),)

★

print(np.where(c > 25, 1, 0)) → [0 0 1 1 1]

Where

↳ if condition True put 1 if false put 0

$d = \text{np.array}([-5, 0, 10, 20])$

$\text{print}(\text{np.clip}(d, 0, 15)) \rightarrow [0 \ 0 \ 10 \ 15]$

clipped or ~~blinded~~ items

to zero ~~for~~ 15 (0, 15)

ex -5 cannot be in 0 to 15
but close of to zero
so put 0

20 cannot be in 0 to 15
but close to 15 so put 15

10) Main functions

$a = \text{np.array}([1.2345, 1.5, 1.8])$

$\text{print}(\text{np.floor}(a)) \rightarrow [1. \ 1. \ 1.]$

$\text{print}(\text{np.ceil}(a)) \rightarrow [2. \ 2. \ 2.]$

$\text{print}(\text{np rint}(a)) \rightarrow [1. \ 2. \ 2.]$ (round to ~~nearest~~
nearest integer)

$\text{print}(\text{np.round}(a, 2)) \rightarrow [1.22 \ 1.5 \ 1.8]$ (round to N decimal
places in this case 2 places)

⇒ other supported functions are .exp & .log & .sqrt &
.sin & .cos etc.

Basic Linear Algebra (dot, inv, determinant, eigen)

$a = \text{np.array}([[1, 2], [3, 4]])$

$b = \text{np.array}([[5, 6], [7, 8]])$

$\text{print}(a @ b)$

$\text{print}(\text{np.dot}(a, b))$

$\rightarrow \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$

★
DOT
Product

$\text{print}(\text{np.linalg.det}(a)) \rightarrow -2.00 \dots$ (Determinant)

$\text{print}(\text{np.linalg.inv}(a)) \rightarrow \begin{bmatrix} -2. & 1. \\ 1.5 & -0.5 \end{bmatrix}$ (Inverse)

$\text{print}(\text{np.linalg.eig}(a)) \rightarrow \text{Eigresult}(\text{eigen values} = \text{array}([-0.3, 5.372 \dots]), \text{eigenvectors} = \text{array}(\begin{bmatrix} [-0.82 \dots, -0.415 \dots], [0.565 \dots, -0.909 \dots] \end{bmatrix}))$

★
eigen value & vector

$\text{np.cross}(a, b) \rightarrow \text{cross product}$

$\Rightarrow$ for 2D numpy arrays, dot product is also same as matrix product

$\Rightarrow$ dot product is also called inner product

$\Rightarrow$ pair wise or element wise mult. is outer product

★
Eigen value
4

Eigen vectors

★ So, reshape like `a.reshape()` only works if `a` is a numpy array would not work if `a` is a list if want to make or use reshape on list do.

`np.reshape(listname, (dimensions))` — for a list etc.
`a.reshape(dimensions)` — for
numpy array. (a) (np is alias for numpy)

★ `.shape` could also be used to change dimensions.

ex.
$$a = \begin{bmatrix} [1 & 2 & 3] \\ [4 & 5 & 6] \end{bmatrix}$$

$a.shape = (2, 3)$

$a \Rightarrow \begin{bmatrix} [1 & 2] \\ [3 & 4] \\ [5 & 6] \end{bmatrix}$

$\Rightarrow$ `.shape` can be used to get dimensions if used as `print(a.shape)` but can also be used to assign dimensions.

$\Rightarrow$ difference b/w `shape` & `reshape` is `shape` gives the dimensions can be assigned dimensions too but `reshape` always creates new array without changing original.

★ $\Rightarrow$ for `.rows()` & `.cols()` inside then dimension are set by ex $(2, 2, 3, 4)$ so make 2 blocks of 2 blocks of 3 rows of 4 colⁿ so last two will always be row then colⁿ & before that blocks will be there, also dimensions can also be given to them by iterable like list.